Conversion Propensity Model

Real-Time Personalized Sales Offers

Team: IA Y DATOS NORTE

Team members: Alejandro Becerril, Sebastián Hernández, Octavio Quintero, and Alan Cervantes

Academic affiliation: Master's in Artificial Intelligence and Data Analytics, Universidad Autónoma de Ciudad Juárez

Problem statement

E-commerce experiences often treat visitors similarly even though their intent and likelihood to convert vary. A visitor who is already close to purchasing may not need a discount, while another visitor may benefit more from product guidance, relevant content, or a carefully selected incentive. Applying discounts broadly can reduce margin and subsidize purchases that would have happened anyway; showing the same content to every visitor can also miss opportunities to address different needs.

The project addresses this problem by estimating conversion propensity from session-level behavioral and contextual signals, then translating the score into a separate decision layer. The model supports prioritization and personalization; the business rules determine which experience to show. This separation makes it possible to tune offer policy without retraining the classifier every time a threshold or campaign rule changes.

Project overview

The solution estimates the likelihood that an e-commerce session will convert and uses a decision layer to select a suitable experience. High-propensity sessions can be shown relevant products without automatically applying a discount; medium-propensity sessions can be considered for a controlled incentive; and low-propensity sessions can receive personalized content or recommendations. The intent is to make offers more relevant while managing incentive cost.

Potential business impact

If validated through experimentation, this approach could support:

- **More efficient discounting:** reduce unnecessary incentives for sessions likely to convert without them.
- **Improved conversion opportunities:** identify sessions where tailored content or an incentive may help remove friction.
- **More relevant experiences:** match product recommendations and messaging to behavioral signals and session context.
- **More measurable marketing decisions:** compare strategies using conversion rate, revenue, offer acceptance, average order value, and contribution margin.
- **A reusable decision framework:** separate propensity scoring from offer rules so the business can test thresholds and strategies independently.

These are potential benefits, not measured outcomes from the prototype. Propensity estimates likelihood, but does not prove that a specific offer causes conversion. An A/B test or other controlled experiment is needed to measure incremental impact and profitability.

Technical implementation

Model development was performed in JupyterLab using Python and scikit-learn. The trained solution is a pipeline with two principal stages: **preprocessor** and **classifier**. The preprocessing stage prepares categorical and numerical fields for the estimator; the classifier is a Random Forest model. Keeping these steps in one fitted pipeline ensures that inference uses the transformations learned during training rather than recreating them separately in the API.

The pipeline uses 15 session-level features: country, city, device category, browser, first-user default channel group, sessions, engaged sessions, bounce rate, average session duration, views, views per session, events per session, item-view events, add-to-carts, and checkouts.

The reported evaluation accuracy was **86%**. Accuracy represents the share of evaluated examples classified correctly. For a fuller evaluation, it should be reviewed with class-specific precision and recall, a confusion matrix, and probability calibration—especially because the business decision layer uses probability thresholds.

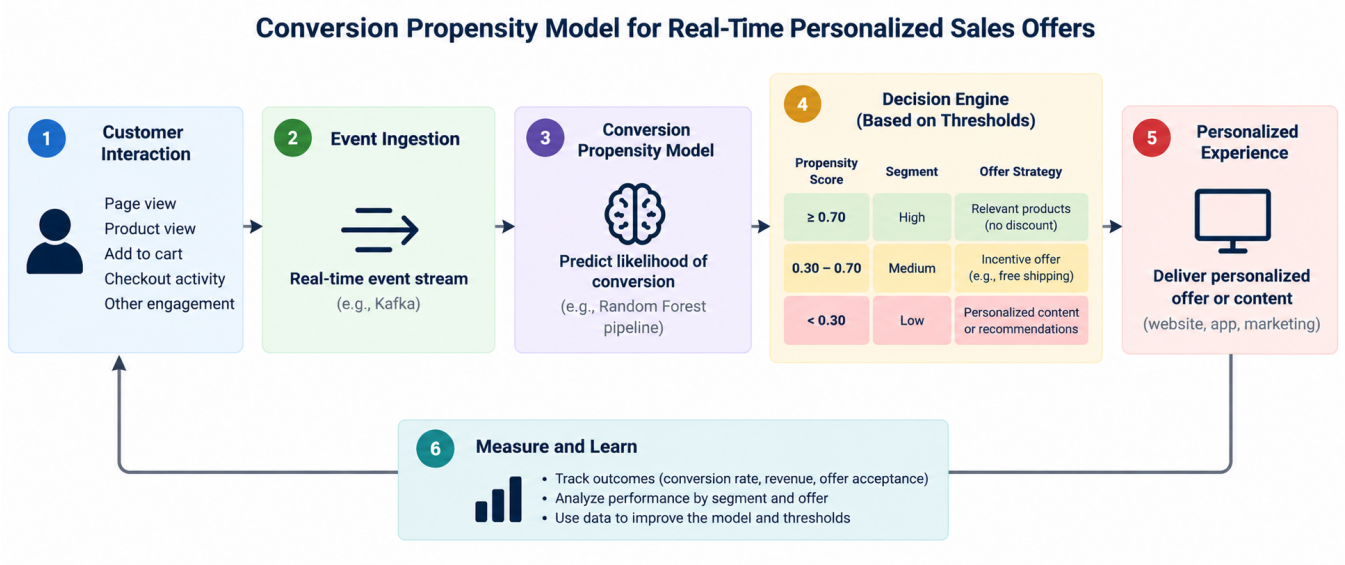
Training workflow

1. Prepare session-level records and define the target label. 2. Separate input features from the target and create an evaluation split. 3. Fit the preprocessing and classifier pipeline on training data. 4. Evaluate predictions against held-out labels. 5. Serialize the fitted pipeline with `joblib` for reuse during inference.

Solution architecture and decision flow

Business solution flow

Customer interactions provide signals such as page views, product views, add-to-cart activity, checkout activity, and other engagement. Session-level features feed the propensity model. A separate decision layer maps the score to an experience, and outcome measurement supports future refinement.



Illustrative thresholds: high propensity (≥0.70) prioritizes relevant products without a discount; medium propensity (0.30–0.70) may receive a controlled incentive such as free shipping; low propensity (<0.30) emphasizes personalized content or recommendations. Thresholds should be validated with business outcomes.

Demo architecture

The demo inference path is shown separately below. A client sends session features in JSON to a FastAPI endpoint. The endpoint validates the request and passes the features to the saved preprocessing and Random Forest pipeline. The score is then passed to the decision layer, which returns a propensity segment and suggested experience.

Demo Architecture — Model Inference API

1. Demo client / request

Send session features as a JSON payload.

↓

2. FastAPI endpoint

Receive request, validate fields, and structure model inputs.

↓

3. Preprocessing pipeline

Transform categorical and numerical features using the fitted preprocessing steps.

↓

4. Random Forest classifier

Generate class prediction and conversion-class probability estimate.

↓

5. Decision layer

Apply business thresholds to assign a propensity segment and offer strategy.

↓

6. API response

Return score, segment, and suggested personalized experience as JSON.

Random Forest decisioning overview

A Random Forest is an ensemble of decision trees. Each tree is trained on a bootstrapped sample of training records and considers a random subset of features when evaluating candidate splits. Because the trees learn different patterns, combining their outputs generally produces a more robust prediction than relying on a single tree.

Decision-tree visualization

The supplied tree visualization illustrates one decision tree from the model. Internal nodes test feature conditions, such as whether a numerical value is below a threshold or whether an encoded category is present. **Gini impurity** indicates how mixed the target classes are at a node; **samples** indicates how many training records reach it; **value** shows the class counts; and **class** identifies the node's predicted class. The forest combines outputs across its trees to produce the final class or probability estimate.

From probability to personalized experience

The fitted pipeline transforms the incoming session using the learned preprocessing steps. The Random Forest then produces a class prediction and class probabilities. The conversion-class probability is used as a propensity score, subject to confirming the positive-class mapping in the model. Business thresholds assign the session to a segment, and the decision layer selects a corresponding experience. The thresholds are policy choices—not properties learned automatically by the classifier.

Demo API architecture

The API exposes the trained pipeline through an HTTP interface. FastAPI defines the request and response endpoints; Pydantic validates request data; Uvicorn runs the application server; pandas structures tabular feature input; and joblib loads the saved pipeline. The response can include the estimated propensity, segment, and recommended experience. The feature names and expected data types should match the training pipeline's input contract.

Potential evaluation and next steps

Beyond the reported accuracy, evaluate precision and recall for both classes, inspect a confusion matrix, and assess whether predicted probabilities are calibrated. Then test the decision policies experimentally. Useful business measures include incremental conversion, revenue per session, average order value, discount or incentive cost, and contribution margin. Compare offer groups with a control group to determine whether an incentive creates additional value rather than simply discounting likely buyers.

Libraries and frameworks used

- **pandas** — tabular data preparation and feature handling.
- **NumPy** — numerical operations and array support.
- **scikit-learn** — preprocessing pipeline, Random Forest classifier, model evaluation, and prediction probabilities.
- **joblib** — saving and loading the fitted model pipeline.
- **FastAPI** — HTTP API endpoints for inference.
- **Pydantic** — request schema and input validation in the API.
- **Uvicorn** — ASGI server used to run the FastAPI application.
- **Matplotlib** — decision-tree visualization, if used to generate the supplied tree plot.